Conclusion

Debriefing
Responsabilité, limites et projection

Fabien Rohrer - Freelance - MSc in Computer Science, EPFL

Debriefing TP #1

- choses où Copilot/IA a vraiment aidé (gain clair)
- moments où ça a dérapé (qualité, incompréhension, perte de temps, ambiguïté)
- règle personnelle que je garde pour la suite

Debriefing TP #2

- Qu’est-ce que l’IA a changé dans votre manière de coder aujourd’hui ?
- Qu’est-ce qui vous prouve que le résultat est correct et maintenable ?
- Qu’est-ce qui pose un risque (données / sécurité / conformité) dans votre façon de l’utiliser ?
- Quelle règle d’équipe vous adoptez dès demain pour une utilisation responsable et durable ?
- Qu’est-ce qui prend de la valeur pour vous, en tant que dev, avec l’IA (architecture, esprit critique, communication, décision) ?

Debriefing TP #3

Garder le contrôle!

- **Ne remplace pas les automatisations existantes** (moins couteuses et plus rigides):
  - Github actions
  - Test suite
  - Automatisations diverses (eslint, etc.)
- L'IA accélère la production. Elle n'accélère pas la vérité.
  - Les erreurs les plus coûteuses sont **plausibles** (style correct, logique fausse).
  - **déplacer l'effort** (moins de frappe clavier, plus de validation).

Debriefing TP #4

Garder le souveraineté!

On garde la main : l'IA propose, l'équipe décide

On protège nos données : pas de secrets dans les prompts

On évite le lock-in : outils interchangeables, standards, sortie de secours

On sécurise la qualité : tests + review, systématiquement

On pilote le coût réel : licence, usage, latence, dépendance fournisseur

Sécurité & confidentialité

Qu’est-ce qui est **interdit** d’envoyer ? (secrets, données client, logs prod, code sensible…)

Qu’est-ce qui est **gris** ? (fragments, structures, pseudo-données)

Qu’est-ce qui est **OK** ? (code non sensible, patterns, exemples synthétiques)

–

prompt injection / fuite d’infos / supply chain / déni de service (coûts) etc.

Coûts

dépendances et économie réelle

Où est le **temps gagné** ? (dev)

Où est le **temps dépensé** ? (revue, debug, incidents)

Quel est le **coût complet** ? licences + latence + gouvernance + formation + risques

Qu’est-ce qui se passe si **le fournisseur change les règles** ou si l’outil est coupé ? (plan B)

Projection développeur

Responsabilité inchangée : le code livré, c'est nous.

On passe de "coder" à "piloter" : donner le bon contexte, poser des contraintes, comparer 2–3 options (l'IDE devient un copilote… parfois "agent").

La valeur remonte en amont : cadrage du problème, critères d'acceptation testables, exemples, specs "exécutables".

La validation devient le cœur du métier : review, tests, lisibilité/maintenabilité, et pas juste "ça compile".

Architecture & découpage deviennent des super-pouvoirs : une base saine + modules clairs → l'IA aide ; sinon elle amplifie le chaos.

Sécurité / data au quotidien : secrets, données sensibles, injection, supply chain… l'IA ajoute de nouveaux risques à maîtriser.

Attention au "faux ROI" : les gains ne sont pas automatiques—sur certains contextes, l'IA peut même ralentir (temps de vérif/corrections). Donc on **mesure**, pas on suppose.

Discussion Ouverte

- Attentes
- Doutes / inquiétudes